

8. Deep Learning 1

Machine learning

Jiri Chmelik

Department of Biomedical Engineering

Introduction

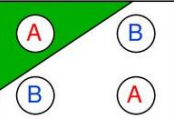
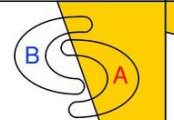
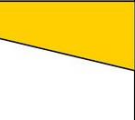
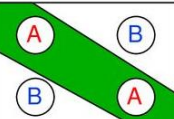
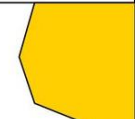
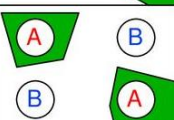
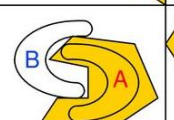
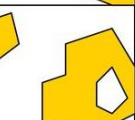
- Differences between shallow and deep ANN
- Differences between ANN and CNN
 - concept
- Basic building blocks
 - Convolution layer
 - Activation layer (ReLU, Softmax)
 - Pooling layer
 - Fully Connected layer
 - DropOut layer
 - BatchNorm layer
- Basic CNN architecture
- Deep learning frameworks

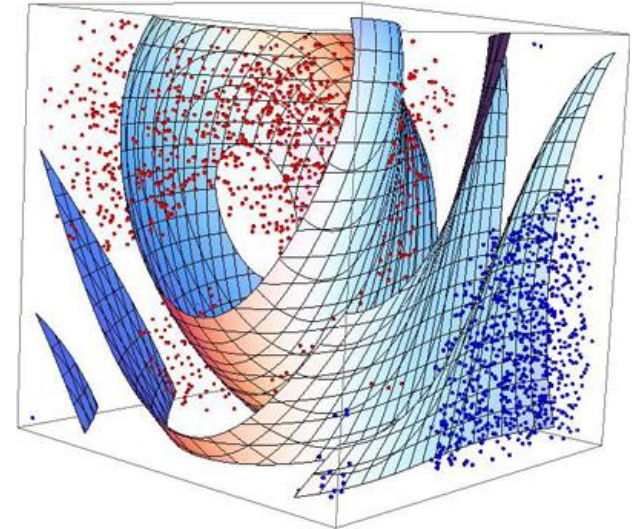
Differences between shallow and deep ANN

- Differences between shallow and deep ANN
- Differences between ANN and CNN
 - concept
- Basic building blocks
 - Convolution layer
 - Activation layer (ReLU, Softmax)
 - Pooling layer
 - Fully Connected layer
 - DropOut layer
 - BatchNorm layer
- Basic CNN architecture

Differences between shallow and deep ANN

- earlier only shallow networks with max. 3 layers were considered (theoretically every N-dimensional convex function should be possible to approximate by 2-layer network (non-convex 3-layer) with enough neurons – division of N-D space by thus hyperplane

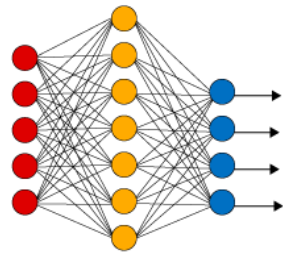
Structure	Types of Decision Regions	Exclusive-OR Problem	Classes with Meshed regions	Most General Region Shapes
Single-Layer 	Half Plane Bounded By Hyperplane			
Two-Layer 	Convex Open Or Closed Regions			
Three-Layer 	Arbitrary (Complexity Limited by No. of Nodes)			



Differences between shallow and deep ANN

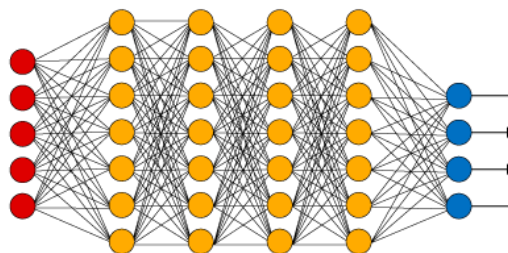
- Nowadays, it is turning out that the deeper architectures brings higher level of abstraction by combining low-level features (formed in first layers) into the high-level features – it brings higher performance than shallow networks.
- Drawback:
 - huge increase of parameters (higher data, memory and computational requirements and overfitting problem)
 - vanishing of activation during forward pass and gradient during backpropagation – can be solved by new techniques (BatchNorm, weight initialization, ReLU activation function)

Simple Neural Network



● Input Layer

Deep Learning Neural Network



● Hidden Layer

● Output Layer

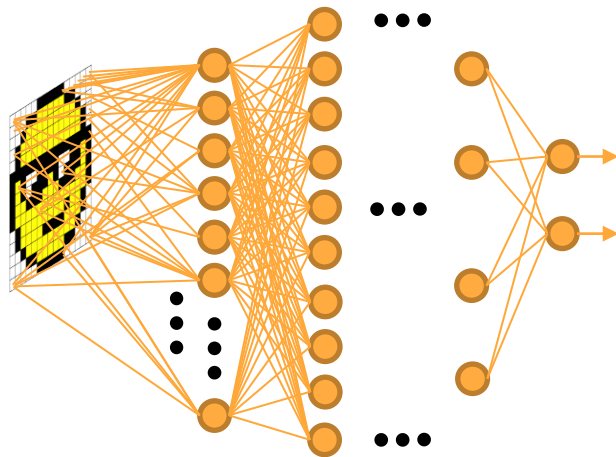
Differences between ANN and CNN

- Differences between shallow and deep ANN
- Differences between ANN and CNN
 - concept
- Basic building blocks
 - Convolution layer
 - Activation layer (ReLU, Softmax)
 - Pooling layer
 - Fully Connected layer
 - DropOut layer
 - BatchNorm layer
- Basic CNN architecture

Differences between ANN and CNN – concept

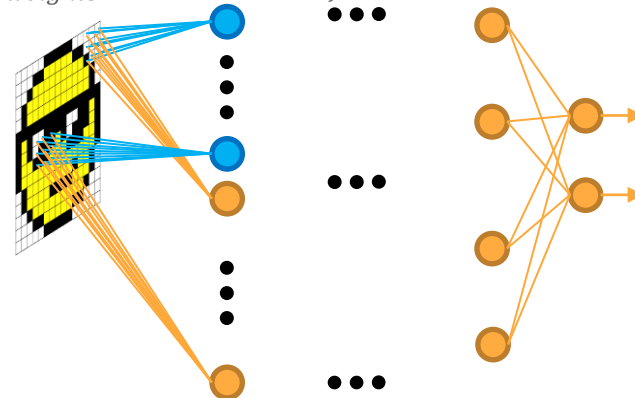
● ANN – fully connected

- each neuron is connected to each neuron from previous layer
- each weight of each neuron is independent
- $N_{weights}^{neuron} = N_{inputs}$ ($200 \times 200 \times 3 = 120,000$)
- $N_{weights}^{layer} = N_{inputs} \cdot N_{neurons}$ ($120,000 \cdot 50 = 6M$)



● CNN

- each neuron is connected to the local neurons from previous layer – space dependency?
- each neuron shares weights with others – want the same local feature over all input
- $N_{weights}^{neuron} = N_{local_conn}$ ($3 \times 3 \times 3 = 27$)
- $N_{weights}^{layer} = N_{local_conn} \cdot N_{features}$ ($27 \times 2 = 54$)

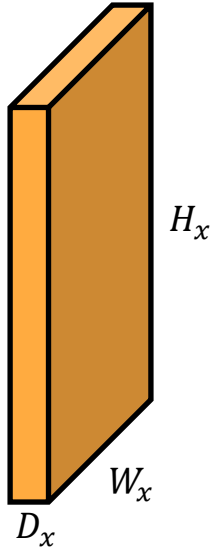


Basic building blocks

- Differences between shallow and deep ANN
- Differences between ANN and CNN
 - concept
- **Basic building blocks**
 - Convolution layer
 - Activation layer (ReLU, Softmax)
 - Pooling layer
 - Fully Connected layer
 - DropOut layer
 - BatchNorm layer
- Basic CNN architecture

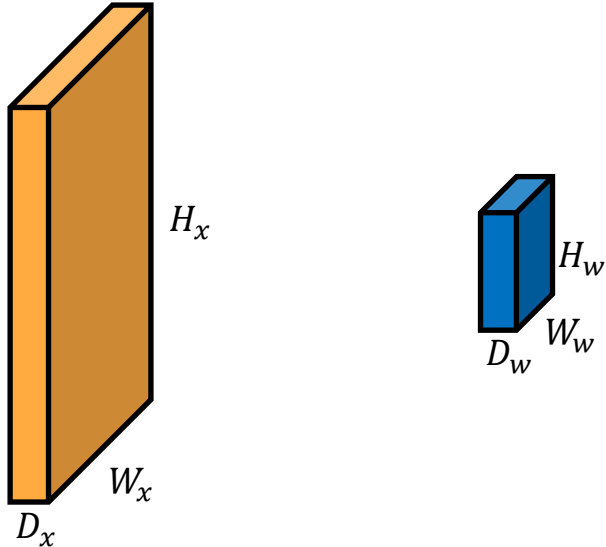
Basic building blocks – Convolutional layer

- input image x with size $W_x \times H_x \times D_x$ – Width, Height, Depth
 - size of input image, depth is number of channels (grayscale, RGB, multimodal image, etc.)



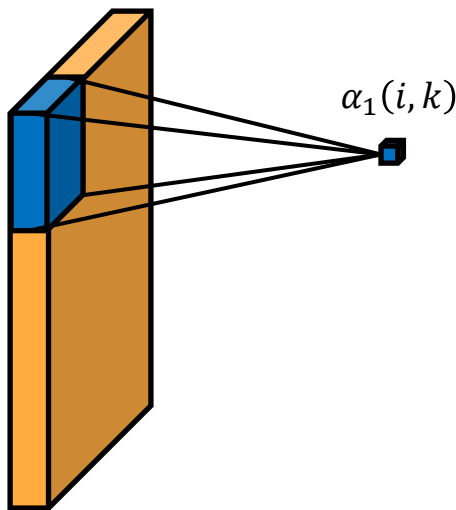
Basic building blocks – Convolutional layer

- filter $\mathbf{w}$ with $W_w \times H_w \times D_w$ – Width, Height, Depth – usually square shape $W_w = H_w = F$
 - size of filter – local connection, depth – always the same as depth of image $D_w = D_x$



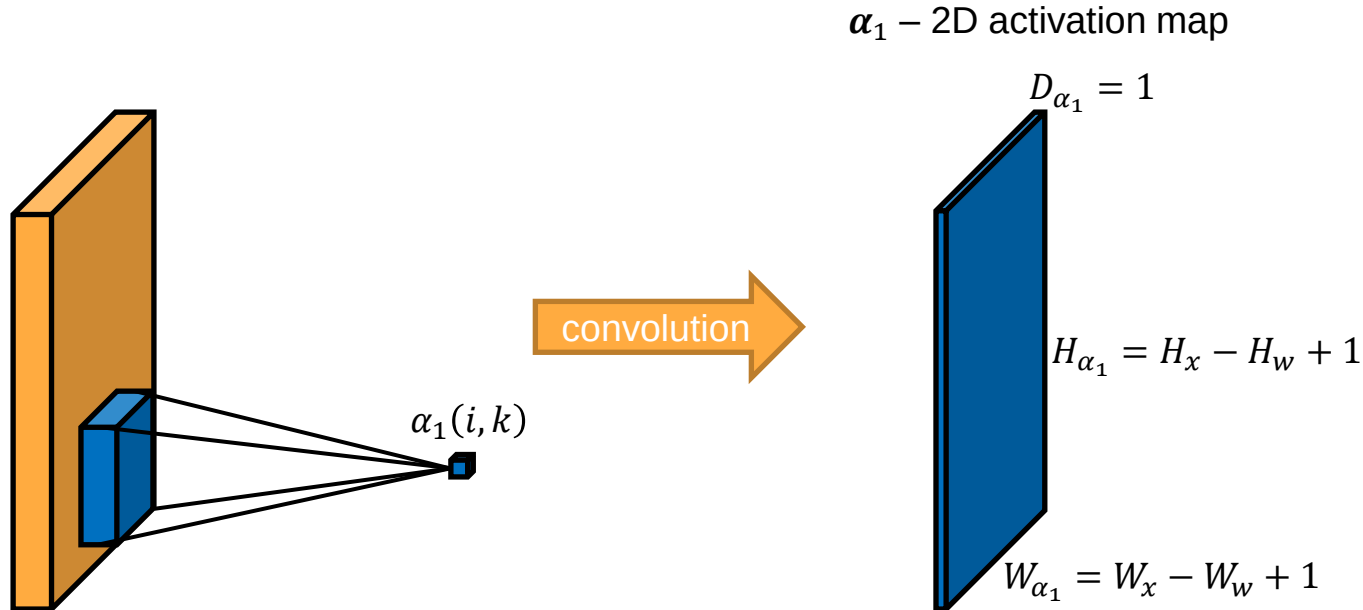
Basic building blocks – Convolutional layer

- compute dot product between part of the input image and filter
 - **convolution** for one image position: $\alpha(i, k) = \sum_{m=-W_w/2}^{W_w/2} \sum_{n=-H_w/2}^{H_w/2} x(i-m, k-n)w(m, n)$
... something familiar?
 - multiplication and final sum – **same as neuron activation**: $\sum_{ind=1}^{W_w H_w} x_{ind} w_{ind}$



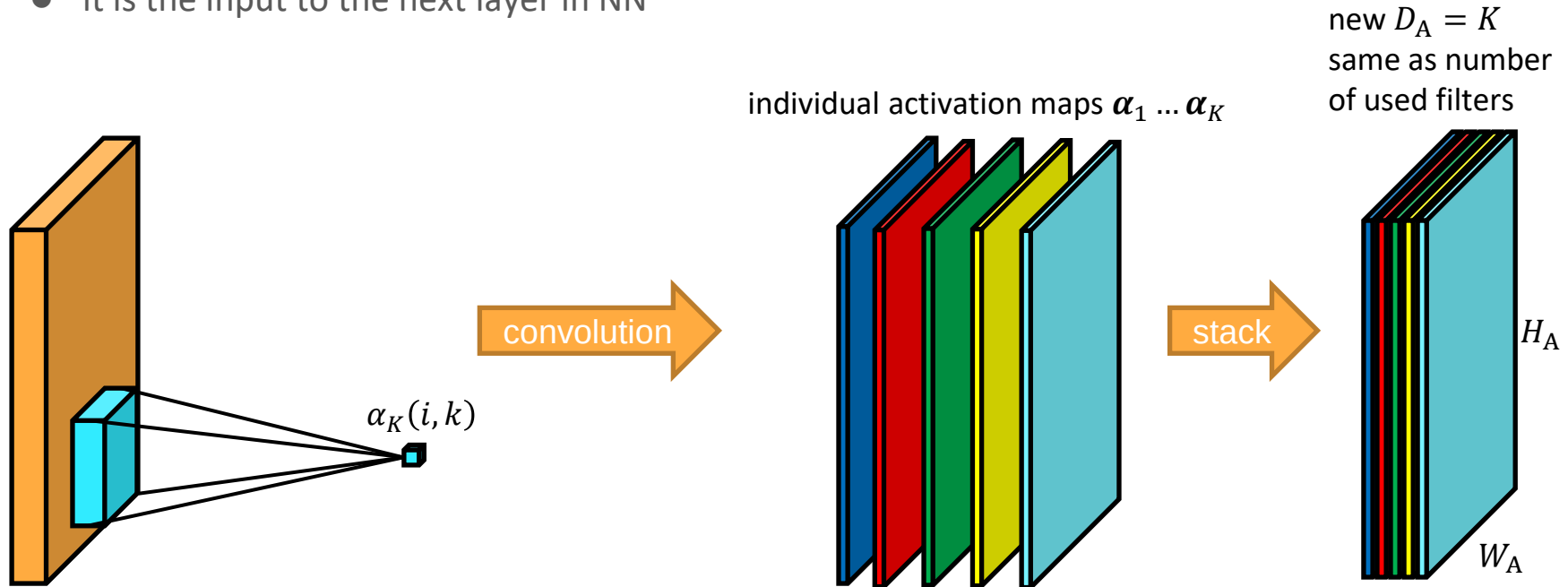
Basic building blocks – Convolutional layer

- compute convolution over all image – activation map of smaller size is obtained



Basic building blocks – Convolutional layer

- compute convolution over all image with another K filters – new activation maps $\alpha_1 \dots \alpha_K$
- output of each convolutional layer is new 3D activation map $\mathbf{A}_l$
- it is the input to the next layer in NN

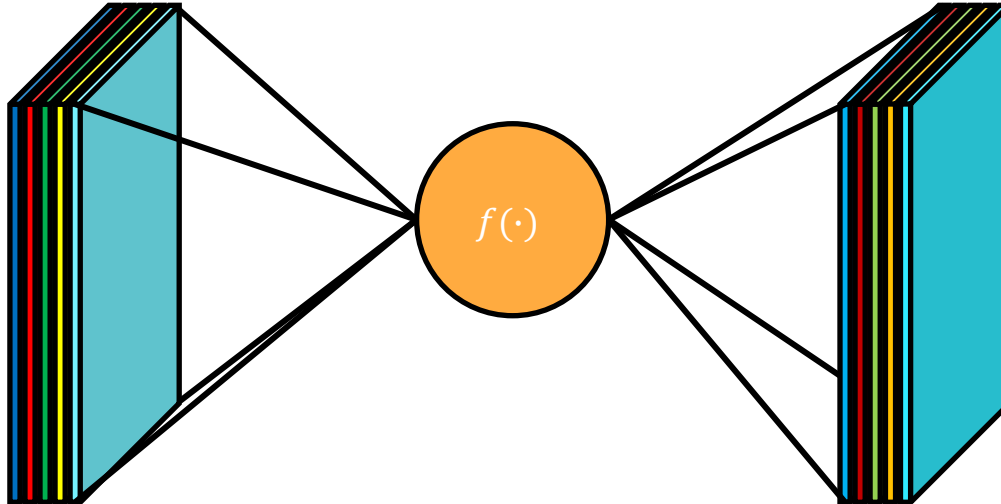


Basic building blocks – Convolutional layer

- Hyperparameters of convolutional layer:
 - number of filters in layer K
 - their spatial extent F (usually odd number)
 - the stride S – step of “convolution”
 - the amount of zero padding P – extension by zeros at border
- Input size $W_l \times H_l \times D_l \rightarrow$ output size $W_{l+1} \times H_{l+1} \times D_{l+1}$
 - $W_{l+1} = \frac{W_l - F_l + 2P_l}{S_l} + 1$
 - $H_{l+1} = \frac{H_l - F_l + 2P_l}{S_l} + 1$
 - $D_{l+1} = K_l$
- All filters in the same layer must have the same size
- Output size must have integer size in each dimension
- Number of parameters per layer is $F_l F_l D_l K_l$ weights + K_l biases

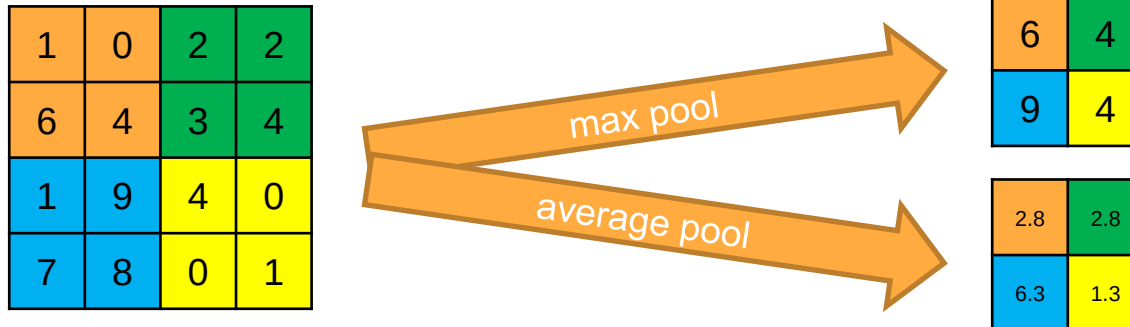
Basic building blocks – Activation layer

- Input: 3D activation map $\mathbf{A}_l$
- Output: 3D feature map of the same size as $\mathbf{A}_l$.
- Same as in ANN – point-by-point transformation of activation by activation function $f(\mathbf{A}_l)$
- Usually ReLU, or Softmax as output layer – without any hyperparameter



Basic building blocks – Pooling layer

- Input: image/activation map/feature map
- Output: shrink input in Weight and Height direction, Depth remains unchanged
- Max pool (more often) or average pool
- Transferring the strongest (max pool) activation on output
- Lower the parameters needed in the deeper layers

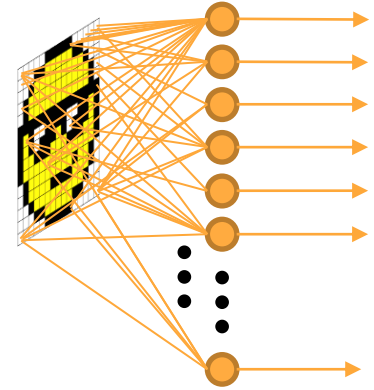


Basic building blocks – Pooling layer

- Hyperparameters of pooling layer:
 - their spatial extent F (usually 2 or 3)
 - the stride S – step moving window (usually 2)
- Input size $W_l \times H_l \times D_l \rightarrow$ output size $W_{l+1} \times H_{l+1} \times D_{l+1}$
 - $W_{l+1} = \frac{W_l - F_l}{S_l} + 1$
 - $H_{l+1} = \frac{H_l - F_l}{S_l} + 1$
 - $D_{l+1} = D_l$
- Output size must have integer size in each dimension
- Number of learned parameters per layer is zero

Basic building blocks – Fully connected layer

- Input: image/activation map/feature map
 - Output: vector or scalar value
 - The same as standard ANN layer
 - Activation function ReLU/Softmax (output layer)
-
- Hyperparameters of fully connected (FC) layer:
 - number of neurons in layer K
 - Input size (vectorized) $W_l H_l D_l \rightarrow$ output size K_l
 - If two subsequent FC layers – $K_{l-1} \rightarrow$ output size K_l
 - Number of parameters in layer $W_l H_l D_l K_l + K_l$



Basic building blocks – DropOut layer

- The same function as in standard ANNs
 - Disable some outputs from neurons in convolutional or fully connected layers
 - Realized as independent layer that is used in the training phase
 - Put before conv. or FC layer
-
- Hyperparameters of DropOut layer:
 - probability that neuron will not be disabled p
 - Input size $W_l \times H_l \times D_l \rightarrow$ output size $W_l \times H_l \times D_l$
 - No learned parameter

Basic building blocks – BatchNorm layer

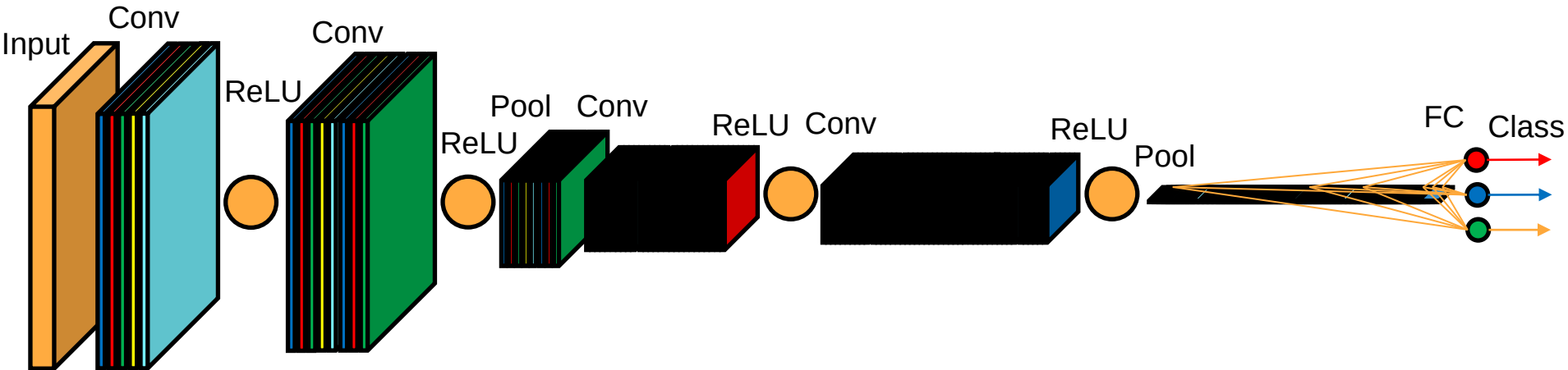
- The same function as in standard ANNs
 - Normalize inputs to the next convolutional or fully connected layer
 - Realized as independent layer – works independently for each filter (feature dimension)
 - Put before conv. or FC layer
-
- Hyperparameters of BatchNorm layer:
 - exponential decay α
 - Input size $W_l \times H_l \times D_l \rightarrow$ output size $W_l \times H_l \times D_l$

Basic CNN architecture

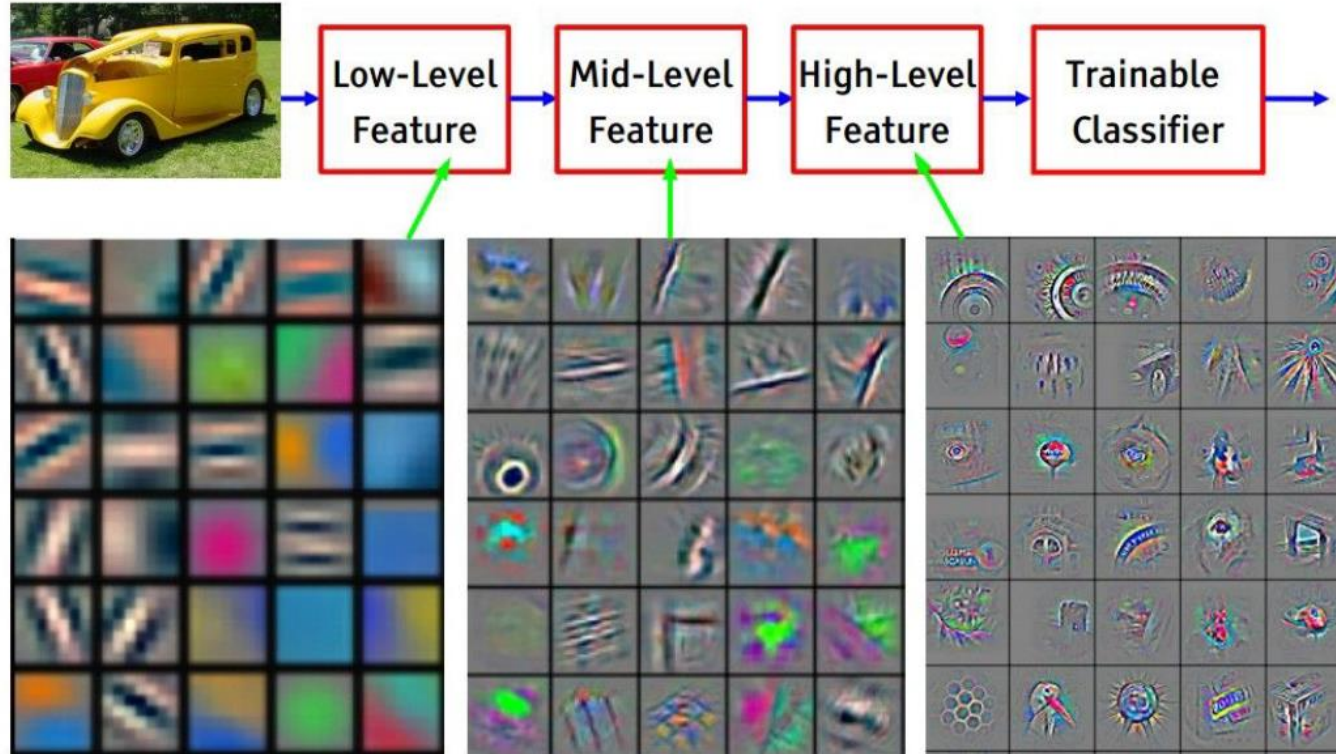
- Differences between shallow and deep ANN
- Differences between ANN and CNN
 - concept
- Basic building blocks
 - Convolution layer
 - Activation layer (ReLU, Softmax)
 - Pooling layer
 - Fully Connected layer
 - DropOut layer
 - BatchNorm layer
- Basic CNN architecture

Basic CNN architecture

- Usually a few combination of convolution and ReLU activation layers is followed by max pooling layer – this is repeated a few times – at the end, 1-3 fully connected layers are used, when the first with ReLU activation functions and the last output FC layer with Softmax activation function.
- Generally, as the input image goes through CNN, its size is gradually reduced in the Width and Height directions, while the Depth is growing – at the end 1D feature vector is classified by output FC layer.



Basic CNN architecture – feature visualization



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Basic CNN architecture – filters and activation map

